

Library Management System

Building the First Lightning Web Component (LWC)

Salesforce Interview Readiness Bootcamp — Day 4

Topic: Lightning Web Components — From Database to Screen

Component Built: **libraryHome**

Environment: Salesforce Developer Edition Org

Tooling: Visual Studio Code with Salesforce CLI (sf)

Project Report

Prepared for Submission

GitHub Repository: github.com/ajayogirala/library-management-system

Table of Contents

1. Project Overview	3
1.1 Objective	3
1.2 Architecture Overview	3
2. Part 1 — Pre-Implementation Analysis	3
3. Part 2 — Understanding the Structure of an LWC	3
4. Part 3 — Hands-On Activity 1: Creating the First LWC	4
4.1 HTML Template	4
4.2 JavaScript Controller	4
4.3 Metadata Configuration	5
4.4 Deployment to Lightning App Builder	6
5. Part 4 — Hands-On Activity 2: Data Binding with Variables	6
6. Part 5 — Hands-On Activity 3: Handling a Button Click	8
7. Part 6 — Hands-On Activity 4: Toggling Status with JavaScript	9
8. Consolidated Final Component Code	11
9. Part 7 — Mini Project Enhancement	12
10. Final Deployment and Live Testing	13
11. Submission Details	14
12. Interview Preparation Notes	14
13. Conclusion	15

1. Project Overview

Over the previous sessions of the bootcamp, the backend of the Placement / Library Management System was completed: the data model was designed, SOQL queries were written, Apex triggers were created, and business rules were automated. However, none of this logic could yet be used by an actual end user, since no visual interface existed. This report documents Day 4 of the bootcamp, in which the first Lightning Web Component (LWC) — named **libraryHome** — was designed, coded, deployed, and tested inside a Salesforce Developer Edition org, taking the application from the database layer all the way to the screen.

1.1 Objective

The goal of the day was to understand what Lightning Web Components are, build a working component from scratch, bind JavaScript variables to the HTML template, respond to button-click events, and deploy the component onto a Lightning Page so that it could be viewed and used inside a live Salesforce application.

1.2 Architecture Overview

Salesforce applications are structured in layers, with the user interacting only with the topmost layer:

- **User** — interacts with the visual interface.
- **Lightning Web Component** — renders the UI and handles events (built today).
- **Apex Classes** — contain server-side business logic (to be connected in a future session).
- **SOQL** — queries records from the Salesforce database.
- **Salesforce Database** — stores the underlying data.

Today's work focused entirely on the top layer of this stack — the Lightning Web Component — with hard-coded values standing in for data that will later be supplied by Apex.

2. Part 1 — Pre-Implementation Analysis

Before writing any code, the following conceptual questions were considered to build intuition for why a UI layer is necessary in a Salesforce application:

- Why users need a graphical interface rather than raw data access.
- Why users cannot directly execute SOQL queries against the org.
- Why JavaScript is required inside a Lightning Web Component.
- Which responsibilities belong to the UI layer (layout, display, interaction).
- Which responsibilities must remain in Apex (security, business logic, data access).

3. Part 2 — Understanding the Structure of an LWC

Every Lightning Web Component is made up of three core files, each with a distinct responsibility:

- **libraryHome.html** — page layout, buttons, text, and images.
- **libraryHome.js** — logic, variables, events, and communication.

- **libraryHome.js-meta.xml** — makes the component available in Lightning App Builder and defines where it can be used.

These three files were generated together using the Salesforce CLI and form the foundation for every activity that follows.

4. Part 3 — Hands-On Activity 1: Creating the First LWC

The component was generated from VS Code using the Salesforce CLI command `sf lightning generate component --type lwc --name libraryHome --output-dir force-app/main/default/lwc`, which created the html, JavaScript, meta-xml, and test files for the new **libraryHome** component.

4.1 HTML Template

The template was wrapped in a `<lightning-card>` titled "Library Management System" with an introductory heading, "Welcome to Library Management System", displayed inside the card body.

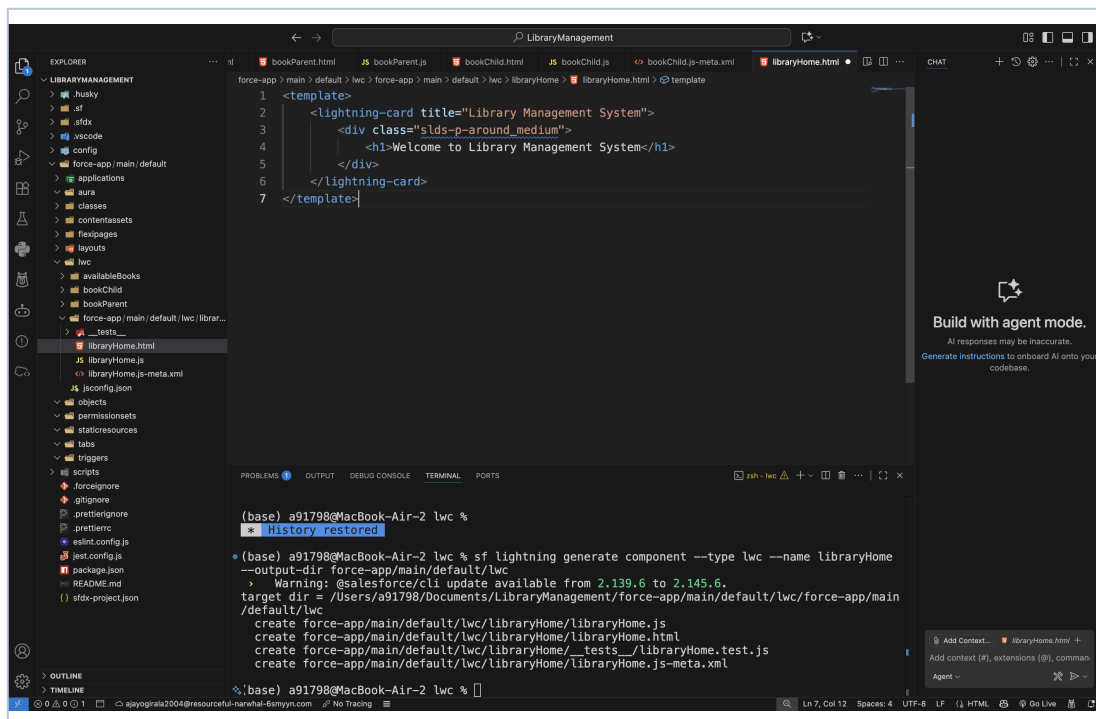


Figure 1: Initial HTML template of the `libraryHome` component, displaying the `lightning-card` title and welcome heading.

4.2 JavaScript Controller

The default JavaScript class generated by the CLI simply extends `LightningElement`, providing the starting point to which variables and methods were added in later activities.

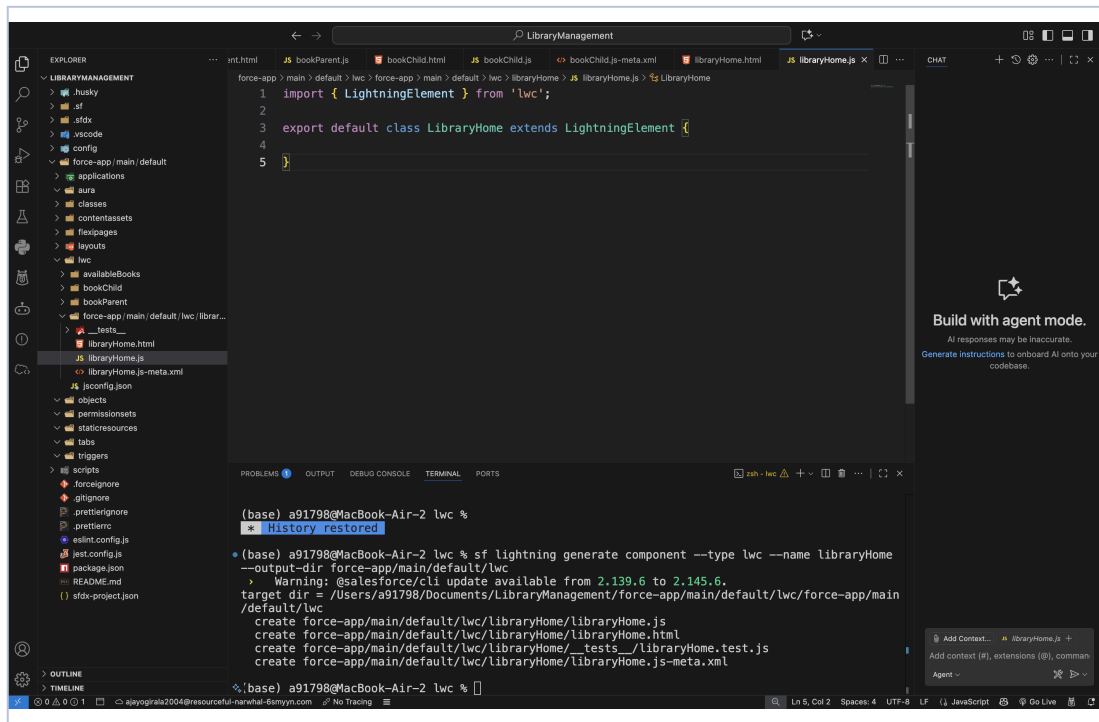


Figure 2: Default libraryHome.js controller stub generated by the Salesforce CLI, extending LightningElement.

4.3 Metadata Configuration

The libraryHome.js-meta.xml file was configured with API version 64.0, isExposed set to true, and target values of lightning__AppPage, lightning__HomePage, and lightning__RecordPage so that the component could be added to Lightning pages through the App Builder.

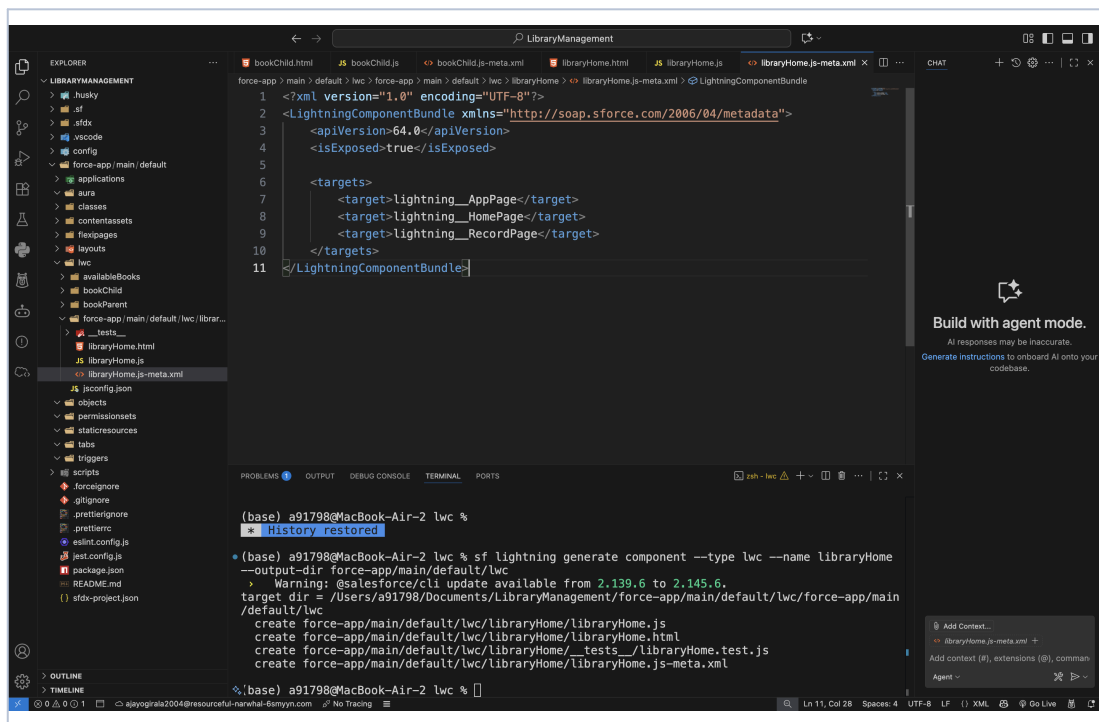


Figure 3: Lightning Component Bundle metadata (js-meta.xml) exposing libraryHome to App, Home, and Record pages.

4.4 Deployment to Lightning App Builder

After deployment to the Developer Edition org, the **libraryHome** component appeared in the Custom components list inside the Lightning App Builder and was dragged onto the "Placement Portal" page, confirming that the component was successfully exposed and deployable.

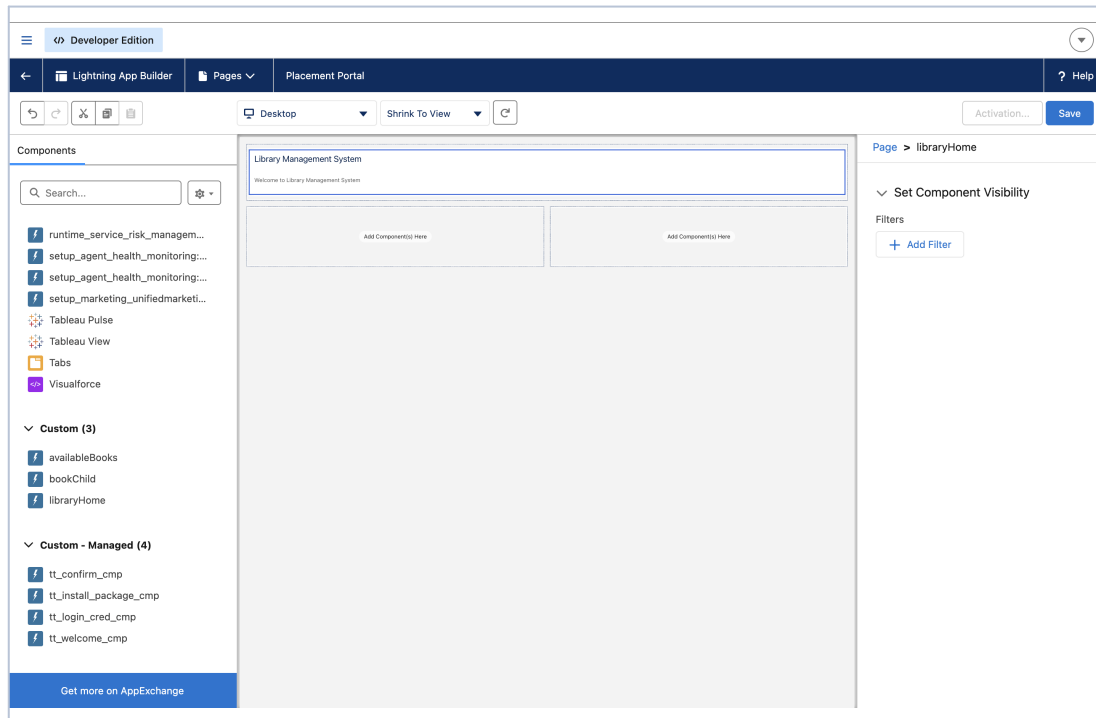


Figure 4: *libraryHome* component listed under Custom components and placed on the Placement Portal page in Lightning App Builder.

5. Part 4 — Hands-On Activity 2: Data Binding with Variables

Three JavaScript class fields — `librarianName`, `libraryName`, and `totalBooks` — were declared and initialised with hard-coded sample values.

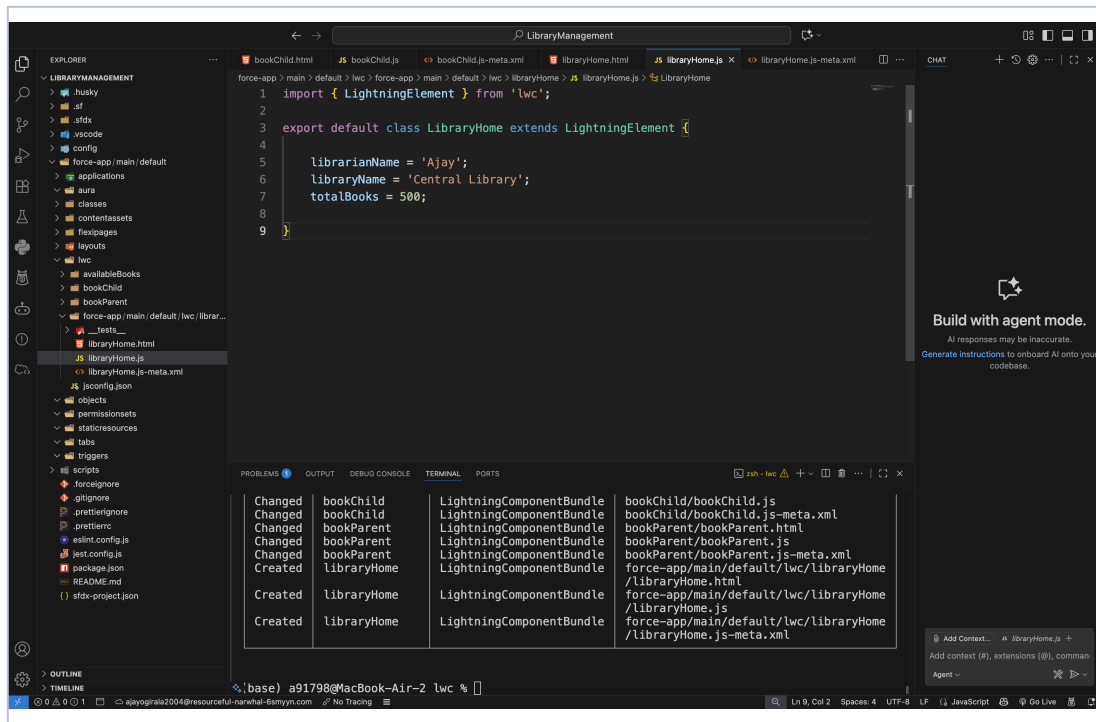


Figure 5: `libraryHome.js` updated with `librarianName`, `libraryName`, and `totalBooks` fields.

These fields were then bound to the HTML template using curly-brace expressions — `{librarianName}`, `{libraryName}`, and `{totalBooks}` — so that any change to a JavaScript value is automatically reflected on screen.

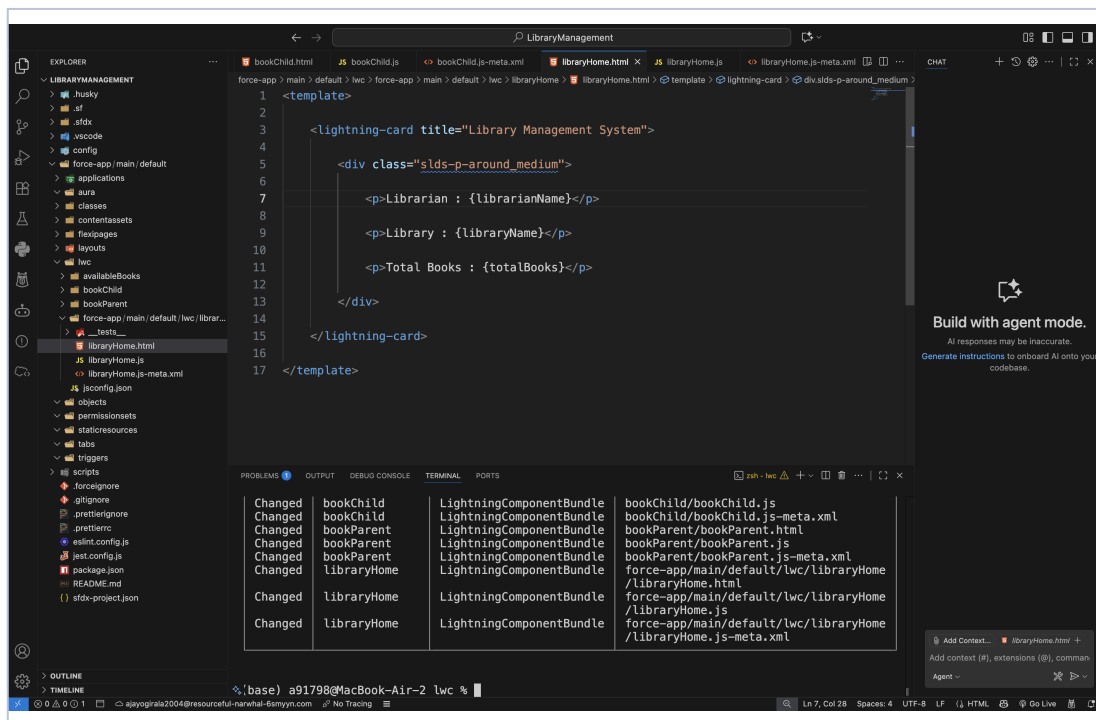


Figure 6: `libraryHome.html` displaying the `librarian`, `library`, and `total-books` values through data binding.

The updated component was redeployed and previewed inside the Lightning App Builder, confirming that the bound values rendered correctly on the page.

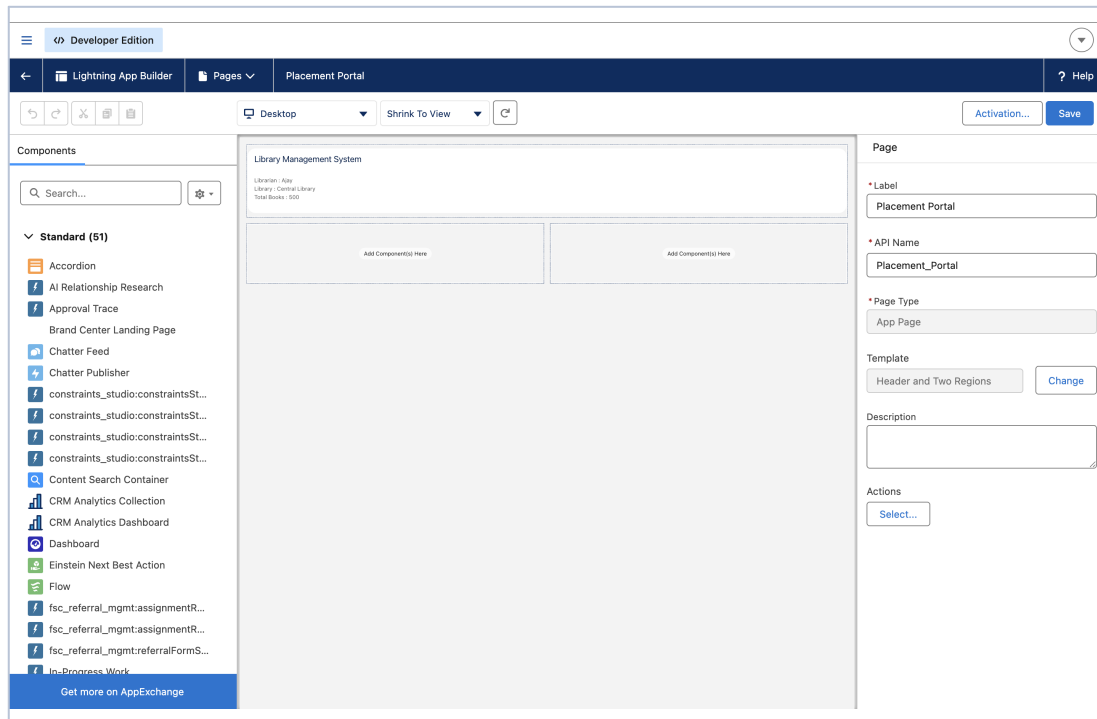


Figure 7: Lightning App Builder preview showing the Librarian, Library, and Total Books values rendered on the Placement Portal page.

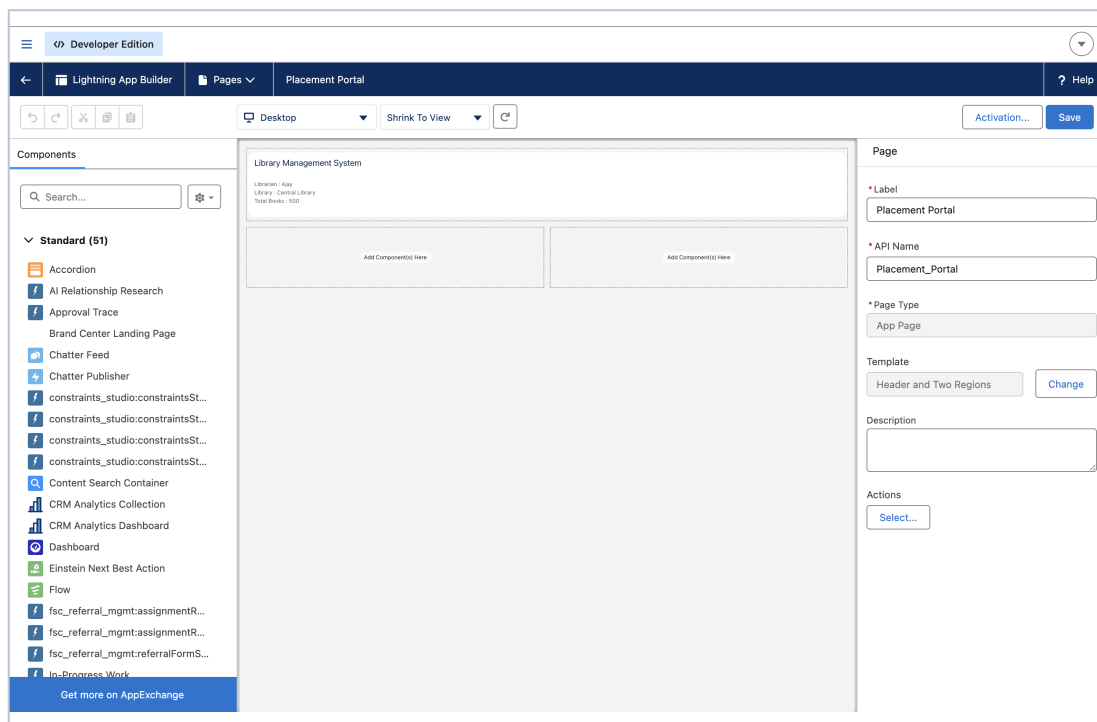


Figure 8: Confirmation preview of the deployed libraryHome component displaying the bound data correctly.

6. Part 5 — Hands-On Activity 3: Handling a Button Click

A message field was added along with a `showMessage()` method that sets `this.message` to "Welcome to Salesforce Library Management" whenever it is invoked.

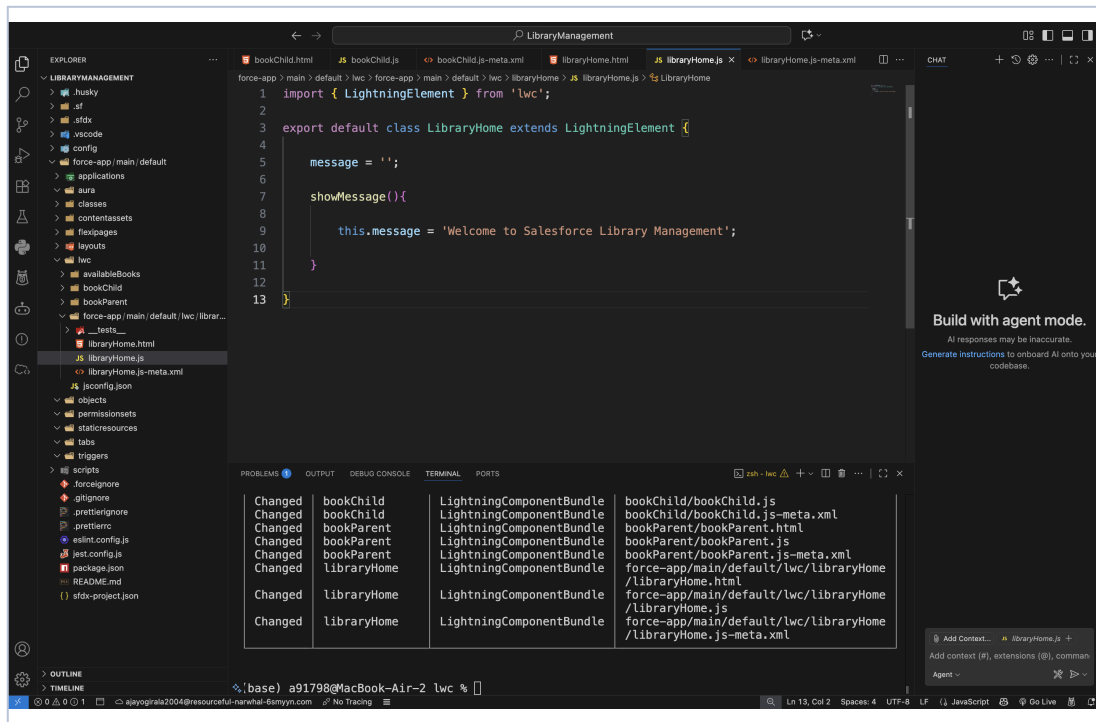


Figure 9: `showMessage()` method added to `libraryHome.js`, updating the `message` field on click.

In the HTML template, a `lightning-button` labelled "Show Welcome Message" was wired to this handler using `onclick={showMessage}`, with a paragraph bound to `{message}` to display the result beneath the button.

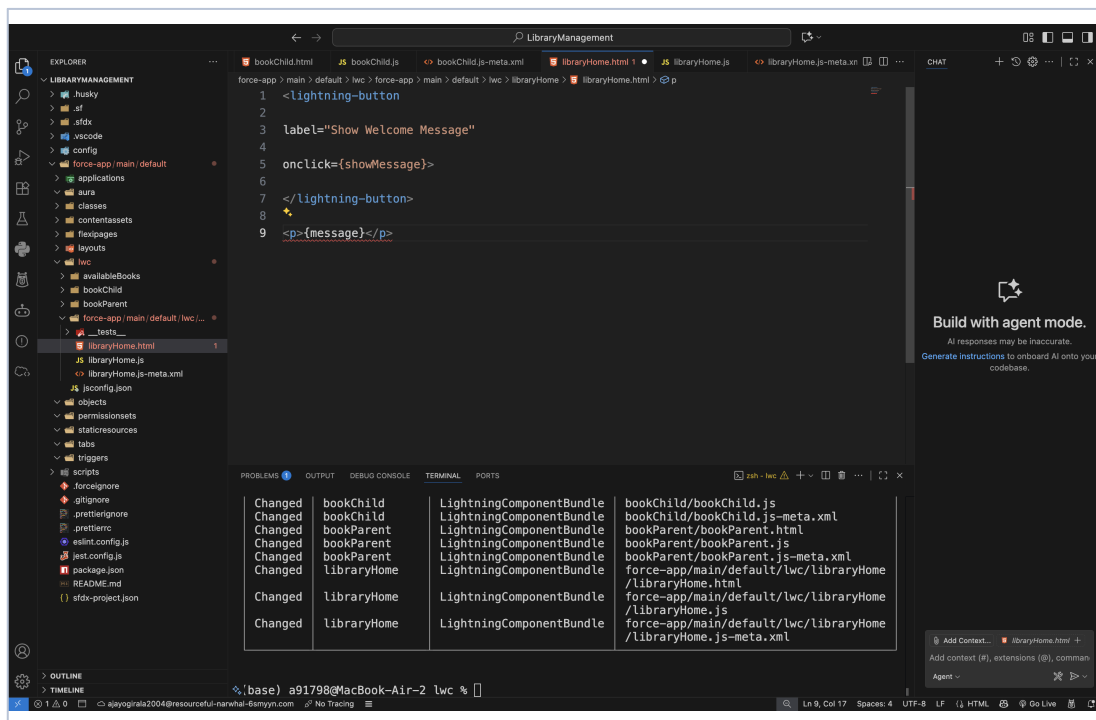


Figure 10: `libraryHome.html` with the "Show Welcome Message" button wired to the `showMessage()` handler and the `{message}` output paragraph.

7. Part 6 — Hands-On Activity 4: Toggling Status with JavaScript

A second field, `status`, was initialised to "Available", and a `changeStatus()` method was added to update it to "Issued" when triggered — entirely on the client side, with no Apex or database interaction involved.

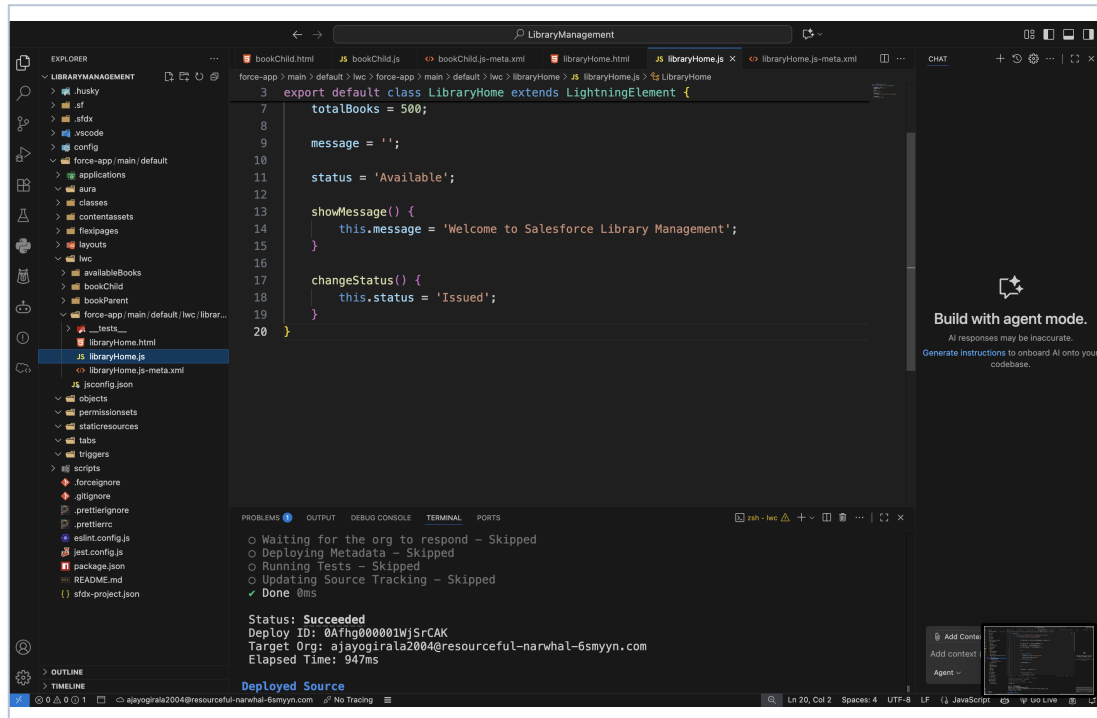


Figure 11: `status` field and `changeStatus()` method added to `libraryHome.js` to toggle book availability.

The template was extended with a status paragraph bound to `{status}` and a second Lightning-button labelled "Issue Book", wired to `onClick={changeStatus}`. The component was then redeployed successfully to the Developer Edition org.

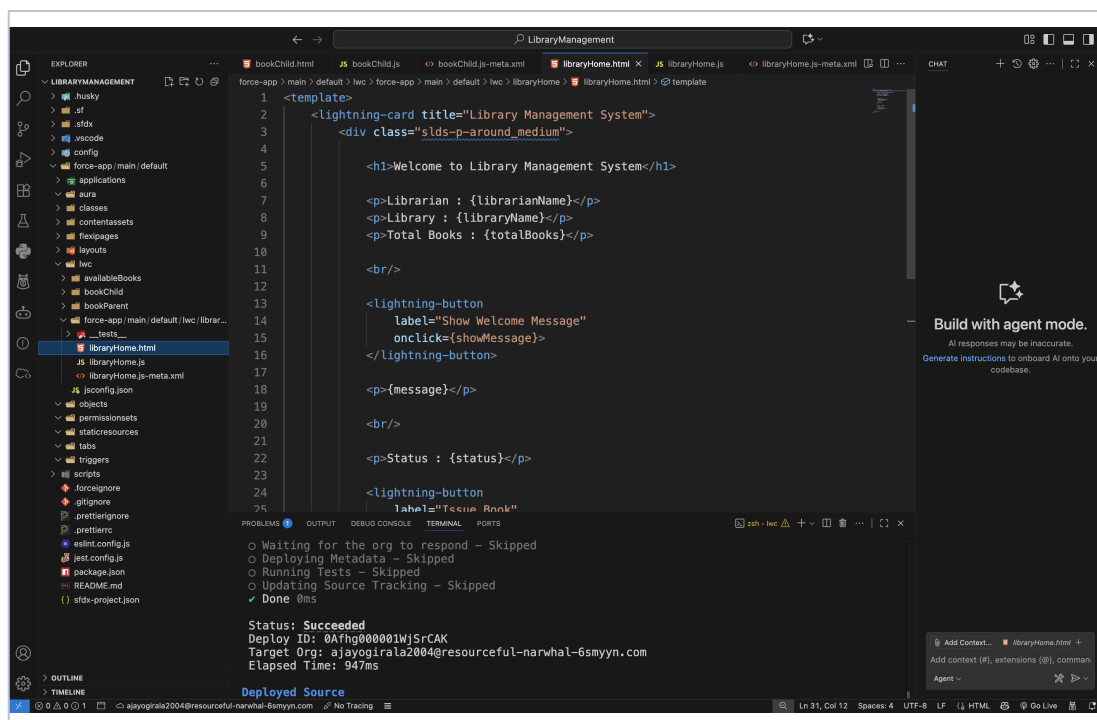


Figure 12: `libraryHome.html` extended with the Status display and "Issue Book" button, together with a successful CLI deployment log.

Previewing the page in Lightning App Builder confirmed that both buttons worked as expected: clicking "Show Welcome Message" revealed the welcome text, while the status line remained ready to toggle to "Issued" once "Issue Book" was clicked.

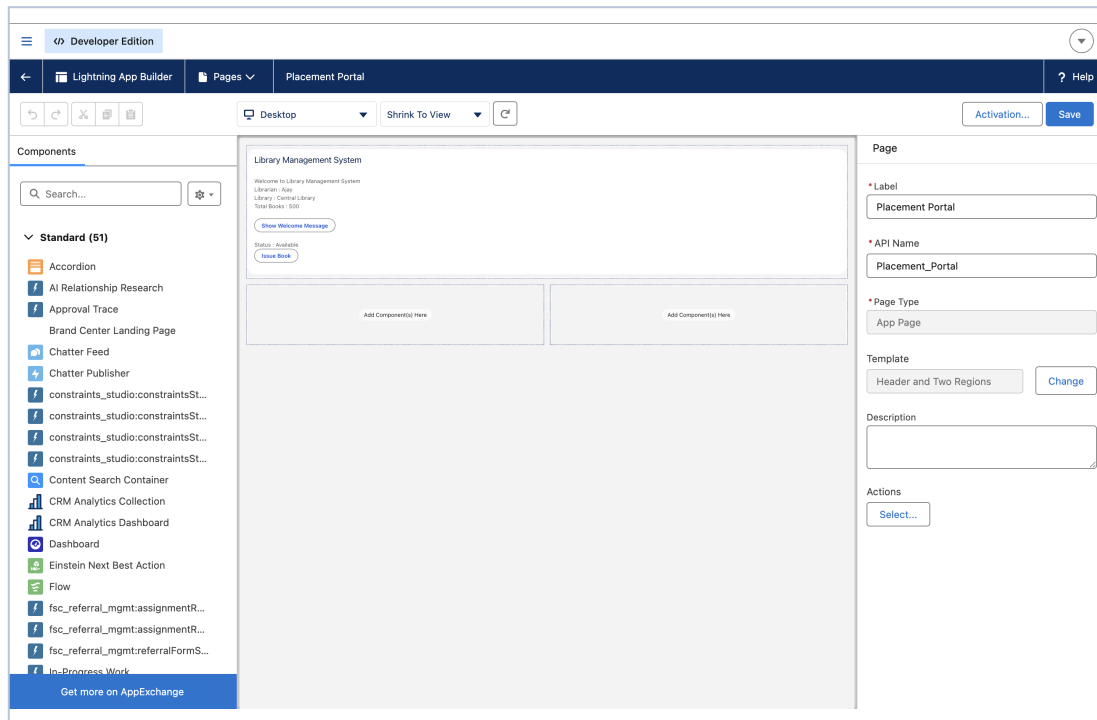


Figure 13: App Builder preview showing the working "Show Welcome Message" and "Issue Book" buttons alongside the Status field.

8. Consolidated Final Component Code

After completing Activities 1 through 4, the **libraryHome** component contained all class fields (librarianName, libraryName, totalBooks, message, status) and both handler methods (showMessage, changeStatus) in a single, consolidated controller, with a matching HTML template rendering every field and button in sequence.

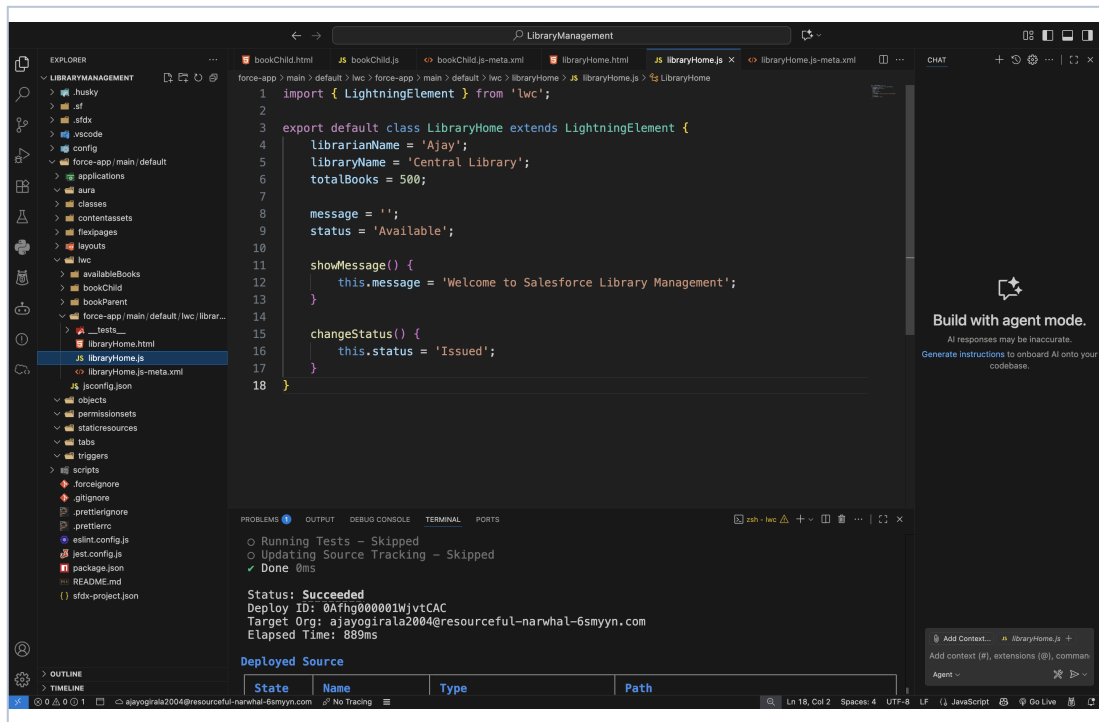


Figure 14: Final consolidated libraryHome.js controller containing all fields and both click-handler methods.

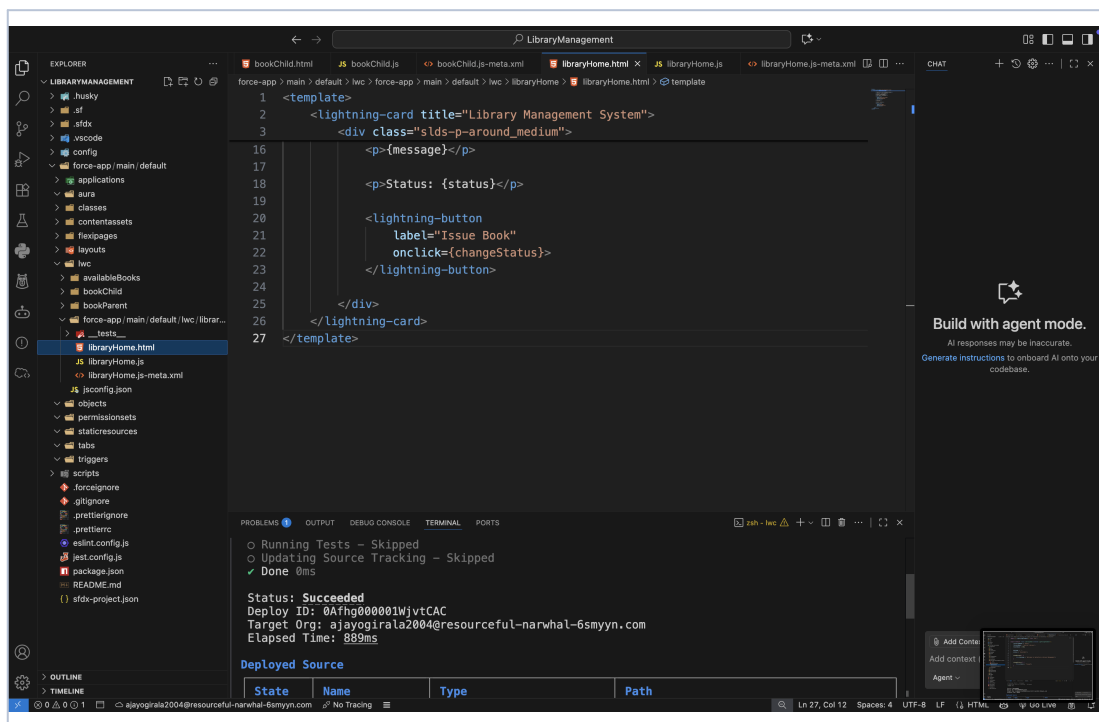


Figure 15: Final consolidated libraryHome.html template rendering all bound fields, the welcome-message button, and the issue-book button.

9. Part 7 — Mini Project Enhancement

The component was enhanced into the first screen of the Placement / Library Management System, adding a "Library Portal" welcome section with the current date, a personalised greeting ("Welcome Ajay"), librarian and department details, and a "Library Statistics" block reporting the number of

books, books issued, and total members — all hard-coded for now, to be replaced by live Apex data in a future session.

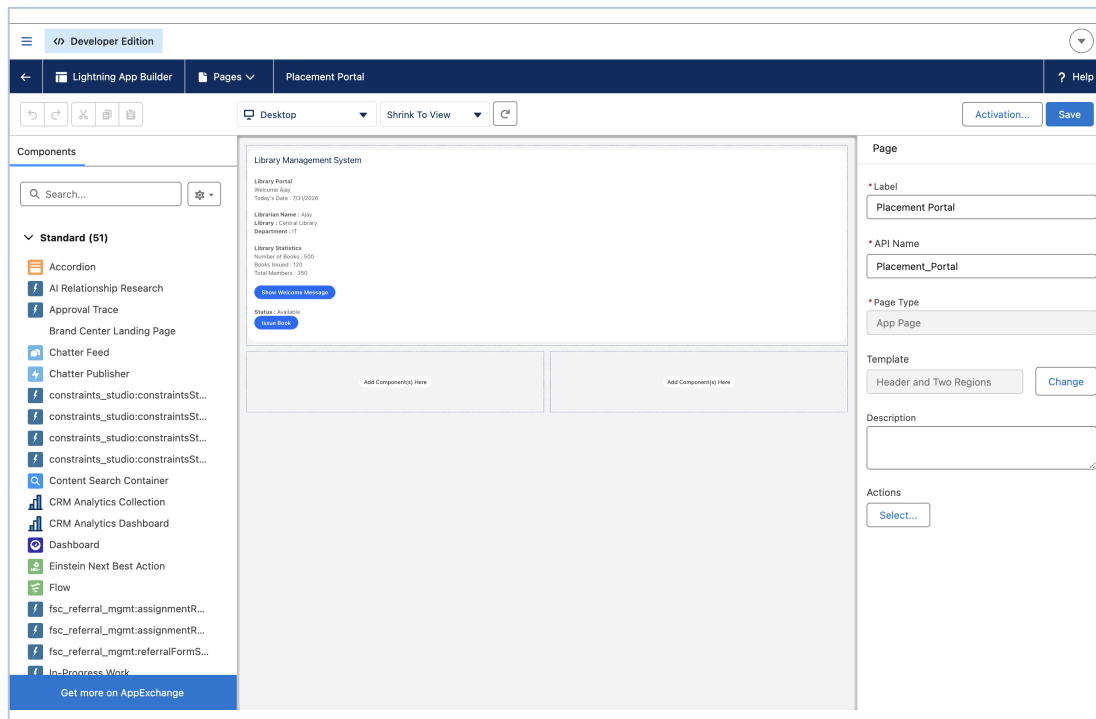


Figure 16: Enhanced Library Portal screen in Lightning App Builder, showing the welcome banner, librarian details, and library statistics.

10. Final Deployment and Live Testing

The completed "Placement Portal" Lightning page was saved and opened directly inside the College Library Management App. Clicking "Issue Book" correctly updated the Status field from "Available" to "Issued", confirming that data binding, event handling, and page deployment were all working end-to-end inside a live Salesforce application.

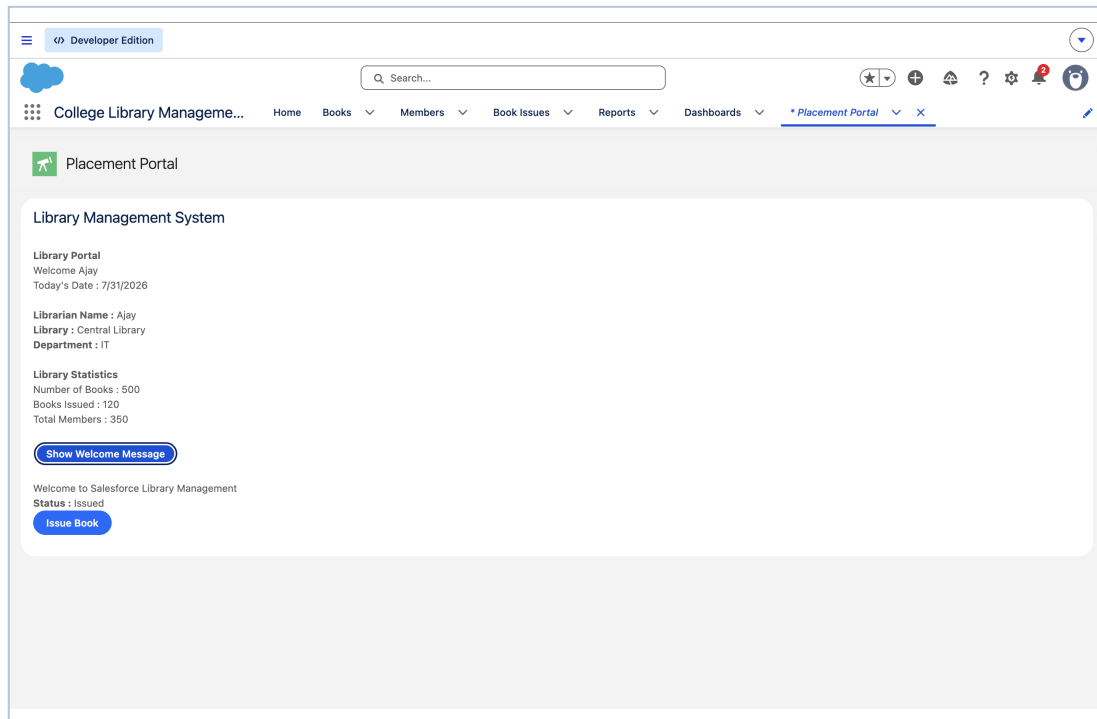


Figure 17: Final live Placement Portal page inside the College Library Management App, with Status updated to "Issued" after clicking Issue Book.

11. Submission Details

The complete source code for the **libraryHome** Lightning Web Component, along with the rest of the Library / Placement Management System project, is hosted in the following GitHub repository:

GitHub Repository: <https://github.com/ajayogirala/library-management-system/tree/main>

The repository contains the full force-app source directory (LWC bundles, Apex classes, objects, and metadata), along with the screenshots and README documentation referenced throughout this report.

12. Interview Preparation Notes

The following questions were prepared as part of today's interview-readiness practice, based directly on the concepts applied while building the libraryHome component:

- 1. What is Lightning Web Components, and why did Salesforce introduce it?
- 2. What is the difference between LWC and Aura Components?
- 3. What are the three files inside an LWC bundle, and what does each contain?
- 4. Why is JavaScript required inside a Lightning Web Component?
- 5. What is data binding, and how was it demonstrated in libraryHome?
- 6. Can an LWC directly execute a SOQL query? Why or why not?
- 7. Why does an LWC eventually need to call Apex?
- 8. Where is a Lightning Web Component deployed and made visible to users?
- 9. How would you explain the libraryHome component built in this session?

A concise, interview-ready definition used throughout this project is: "Lightning Web Components is Salesforce's modern UI framework based on Web Standards (HTML, JavaScript, CSS). It enables developers to build reusable, fast, and maintainable user interfaces that interact with Salesforce data."

13. Conclusion

By the end of Day 4, the **libraryHome** Lightning Web Component evolved from an empty generated stub into a fully functional, deployed page featuring bound data fields, two interactive buttons, and a live status toggle — successfully bridging the gap between the previously built data model and an actual user-facing screen. The values used today were hard-coded; the next phase of the bootcamp will connect this same interface to Apex classes so that the librarian, book, and status information is retrieved from the real Salesforce database.